Operating System Concepts

CPU scheduling · synchronization · memory management · disk scheduling

Chimaroke Amadi · GoMyCode

In this checkpoint I simulated how an operating system manages several tasks at once. For each part I worked through the scenario step by step, drew the charts, and did the calculations. To check my answers, I also wrote small simulations in JavaScript (`simulations.js`), and every number in this report is generated from them.

Summary of results

Part	Algorithm A	Algorithm B	Better
1. CPU scheduling (avg waiting time)	FCFS: 3.33	Round Robin (q=2): 3.33	Round Robin is more responsive (avg response 1 vs 3.33)
2. Synchronization (final counter)	No lock: 109–120 in my runs	Mutex: always 200	Mutex
3. Page replacement (page faults)	FIFO: 6	LRU: 6	A tie for this string, LRU better in general
4. Disk scheduling (head movement)	FCFS: 640	SSTF: 236	SSTF (63% less)

Part 1: Process Scheduling

Process	Arrival time	Burst time
P1	0	5
P2	1	3
P3	2	1

The formulas I used: **turnaround = completion – arrival**, **waiting = turnaround – burst** (the time spent in the ready queue), and **response = first time on the CPU – arrival**.

1.1 First Come, First Served (FCFS)

The CPU runs the processes in the order they arrive, and each one runs until it finishes (non-preemptive). P1 arrives first at time 0 and keeps the CPU until time 5, even though P2 and P3 arrive while it's running. Then P2 runs from 5 to 8, and P3 from 8 to 9.



Process	Arrival	Burst	Completion	Turnaround completion – arrival	Waiting turnaround – burst	Response first run – arrival
P1	0	5	5	5 – 0 = 5	5 – 5 = **0**	0
P2	1	3	8	8 – 1 = 7	7 – 3 = **4**	4
P3	2	1	9	9 – 2 = 7	7 – 1 = **6**	6
Average				**6.33**	**3.33**	**3.33**

> Average waiting time (FCFS) = (0 + 4 + 6) / 3 = 10 / 3 = **3.33** time units

1.2 Round Robin (time quantum = 2)

Each process gets the CPU for at most 2 time units, then goes to the back of the ready queue if it isn't finished. The ready queue changes like this (remaining burst time in brackets):

CPU	At time	Ready queue afterwards
P1 runs 0–2	2	P2(3), P3(1), P1(3)
P2 runs 2–4	4	P3(1), P1(3), P2(1)
P3 runs 4–5	5	P1(3), P2(1)
P1 runs 5–7	7	P2(1), P1(1)
P2 runs 7–8	8	P1(1)
P1 runs 8–9	9	(empty)

At time 2, P1's quantum ends at the same moment P3 arrives. I used the usual convention that the new arrival joins the queue first and the preempted process goes behind it, so the queue becomes P2, P3, P1.



Process	Arrival	Burst	Completion	Turnaround completion – arrival	Waiting turnaround – burst	Response first run – arrival
P1	0	5	9	9 – 0 = 9	9 – 5 = **4**	0
P2	1	3	8	8 – 1 = 7	7 – 3 = **4**	1
P3	2	1	5	5 – 2 = 3	3 – 1 = **2**	2
Average				**6.33**	**3.33**	**1**

> Average waiting time (Round Robin, q = 2) = (4 + 4 + 2) / 3 = 10 / 3 = **3.33** time units

1.3 Which algorithm is more responsive?

Here the average waiting time is the same for both algorithms (3.33), but the waiting is shared very differently. With FCFS, the short process P3 (only 1 unit of work) waits 6 units because it's stuck behind the long P1. This is called the **convoy effect**. With Round Robin, P3 waits only 2 units, and nobody waits more than 4.

Round Robin is more responsive. Responsiveness is about how soon a process first gets the CPU (response time), and the average response time drops from **3.33** with FCFS to **1** with Round Robin. Every process is guaranteed a turn within a short, bounded time, at most (n – 1) × quantum, because the CPU is preempted after each quantum. That's why time-sharing and interactive systems use Round Robin-style scheduling. The cost is extra context switches (5 switches instead of 2 here), and turnaround time doesn't improve (6.33 for both). If the quantum is too large, Round Robin behaves like FCFS. If it's too small, most of the time is spent switching.

> With the other convention (the preempted P1 queued before the newly arrived P3), the order would be P1, P2, P1, P3, P2, P1 and the average waiting time would be 4.00. Round Robin would still have the better response time.

Part 2: Process Synchronization

P1 and P2 share the variable `counter` (starting at 0), and each increments it 100 times. The expected final value is **200**.

2.1 What could go wrong without synchronization

The problem is a **race condition**. `counter = counter + 1` looks like one step, but the CPU actually does it in three: **LOAD** the value into a register, **ADD** 1, and **STORE** it back. The scheduler can switch between P1 and P2 in the middle of these steps. If both load the same value before either stores, one of the increments is lost:

Step	Process	Instruction	Its register	counter in memory
1	P1	`LOAD counter → reg`	0	0
2	P2	`LOAD counter → reg`	0	0
3	P1	`ADD reg + 1`	1	0
4	P2	`ADD reg + 1`	1	0
5	P1	`STORE reg → counter`	1	1
6	P2	`STORE reg → counter`	1	1

Two increments happened, but `counter` only went from 0 to 1. Because this can happen any number of times, the final result is unpredictable and is usually **less than 200** (in theory anything from 2 to 200). It can also change from one run to the next, which makes the bug hard to reproduce. I simulated 5 runs with random switching between the two processes and got **109, 117, 118, 111, 120**, never 200. The part of the code that touches the shared variable is the **critical section**, and only one process should be inside it at a time (**mutual exclusion**).

2.2 Suggested mechanism: a mutex

I'd use a **mutex** (mutual exclusion lock). Before entering the critical section, a process must `lock` the mutex. If the other process already holds it, the caller is blocked until it's unlocked. The lock and unlock operations are atomic (the OS or hardware guarantees they can't be interrupted halfway), so only one process can ever hold it. A mutex fits this problem well because there's exactly one shared resource and the same process that locks it also unlocks it. A binary semaphore initialised to 1 would also work (`wait`/`signal` instead of `lock`/`unlock`), and so would a monitor, which puts the counter and its operations inside one object that only one process can use at a time.

2.3 Pseudocode

```
shared counter ← 0
shared mutex   ← new Mutex()          // unlocked at the start

procedure Worker(name)                // run once as P1 and once as P2, at the same time
    for i ← 1 to 100 do
        // ----- entry section -----
        lock(mutex)                   // waits here if the other process holds the lock

        // ----- critical section: only one process at a time -----
        temp    ← counter             // LOAD
        temp    ← temp + 1            // ADD
        counter ← temp                // STORE

        // ----- exit section -----
        unlock(mutex)                 // lets the waiting process in

        // remainder section (code that doesn't touch counter)
    end for
end procedure

start Worker("P1") and Worker("P2") in parallel
wait for both to finish
print counter                         // always 200
```

The same idea with a semaphore:

```
shared S ← Semaphore(1)
...  wait(S)      // S: 1 → 0, a second caller blocks while S = 0
     counter ← counter + 1
     signal(S)    // S: 0 → 1, wakes one waiting process
```

This solution meets the three requirements of a correct critical-section solution. **Mutual exclusion**: only the holder of the lock can be in the critical section. **Progress**: when the lock is free, a waiting process gets it straight away. **Bounded waiting**: each process unlocks after one increment, so the other is never kept waiting forever. When I added the lock to the same 5 simulated runs, every one gave **200, 200, 200, 200, 200**. The critical section is kept as small as possible (only the increment), so the two processes still run in parallel the rest of the time.

Part 3: Memory Management

The process uses 5 different pages, but only **3 frames** are available. Reference string: **1, 2, 3, 2, 4, 1, 5**. A **F** means a page fault (the page wasn't in memory and had to be loaded), and a ✓ means a hit. The first three references are always faults, because memory starts empty (compulsory misses).

3.1 FIFO (First In, First Out)

When a page has to be replaced, FIFO evicts the page that has been in memory the **longest**, no matter how recently it was used.

Reference	1	2	3	2	4	1	5
Frame 1	1	1	1	1	4	4	4
Frame 2		2	2	2	2	1	1
Frame 3			3	3	3	3	5
Fault?	F	F	F	✓	F	F	F
Evicted					1	2	3

6 page faults, 1 hit out of 7 references.

When page 4 is requested, page 1 is the oldest page in memory, so it's evicted. Then page 1 is needed straight away, so it causes another fault and evicts page 2 (now the oldest). Finally, 5 evicts 3.

3.2 LRU (Least Recently Used)

LRU evicts the page that hasn't been **used** for the longest time, based on the idea that a page used recently will probably be used again soon (locality of reference).

Reference	1	2	3	2	4	1	5
Frame 1	1	1	1	1	4	4	4
Frame 2		2	2	2	2	2	5
Frame 3			3	3	3	1	1
Fault?	F	F	F	✓	F	F	F
Evicted					1	3	2

6 page faults, 1 hit out of 7 references.

When page 4 is requested, page 1 is the least recently used (page 2 was used one step earlier), so 1 is evicted, the same choice as FIFO. When page 1 comes back, LRU evicts 3 instead of 2, because 2 was used more recently. The pages kept are different, but the number of faults ends up the same.

3.3 Comparison

Algorithm	Page faults	Hits	Fault rate
FIFO	**6**	1	86%
LRU	**6**	1	86%
Optimal (for comparison)	5	2	71%

For this reference string, FIFO and LRU perform the same: 6 page faults each. The string is very short and has almost no repeated use. Only page 2 is referenced twice in a short time, and both algorithms keep it. After that, pages 4, 1 and 5 each appear once in a row, so there's no "recent use" pattern for LRU to take advantage of. Even the Optimal algorithm, which knows the future and is the best possible, only saves one fault (5). So there was very little room to improve.

In general, LRU performs better than FIFO. Real programs have locality: they keep using the same pages (a loop, a function, an array) for a while. LRU keeps those recently used pages, while FIFO may throw out a page that is heavily used just because it was loaded early. FIFO can also suffer from **Belady's anomaly**, where giving it more frames causes *more* faults, and LRU never does. The downside of LRU is cost: the system has to record when every page is used (with counters or a stack, or approximately with reference bits as in the Clock algorithm), while FIFO only needs a simple queue.

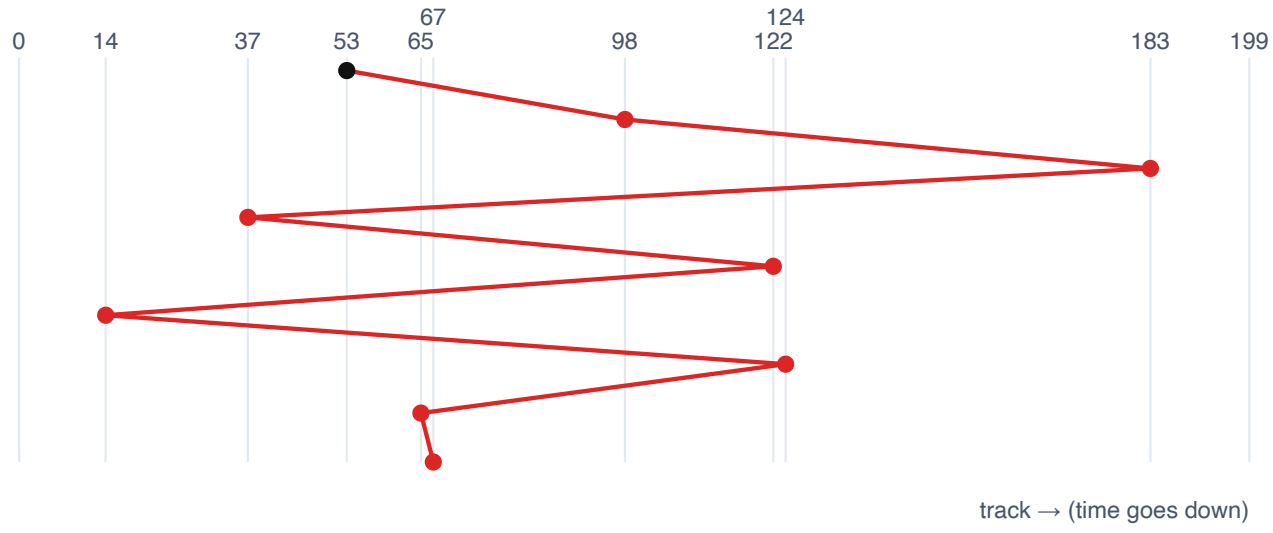
Part 4: Disk Scheduling

The disk head starts at track **53**. Pending requests, in the order they arrived: **98, 183, 37, 122, 14, 124, 65, 67**. Head movement for each step = |destination − current position|.

4.1 FCFS (First Come, First Served)

The requests are served in the order they arrived: **53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67**

#	Move	Distance
1	53 → 98	|98 − 53| = 45
2	98 → 183	|183 − 98| = 85
3	183 → 37	|37 − 183| = 146
4	37 → 122	|122 − 37| = 85
5	122 → 14	|14 − 122| = 108
6	14 → 124	|124 − 14| = 110
7	124 → 65	|65 − 124| = 59
8	65 → 67	|67 − 65| = 2
Total head movement		**640 tracks**

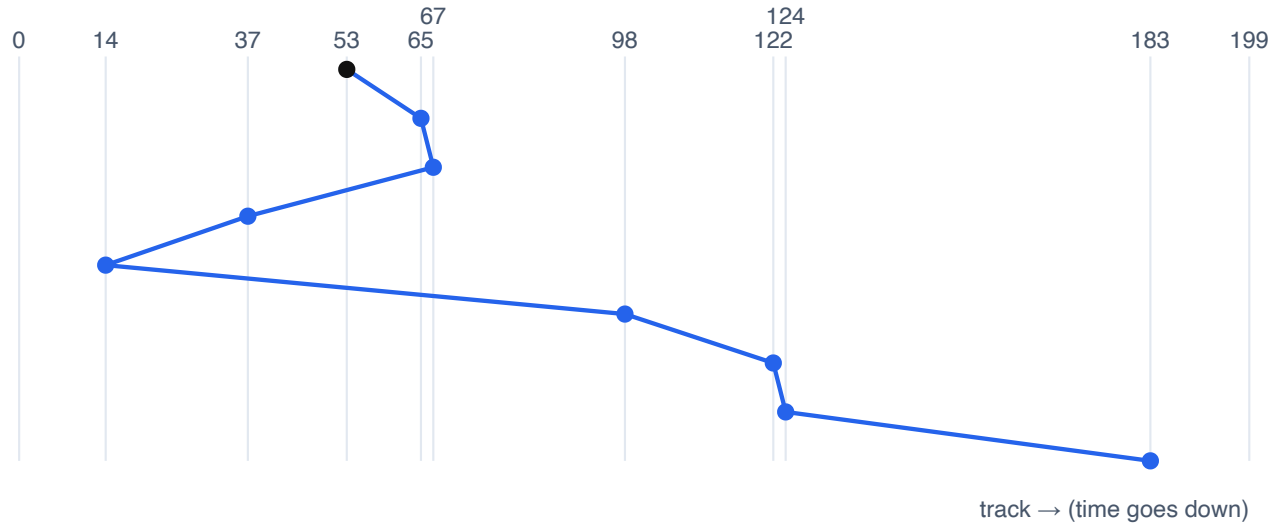


Total head movement (FCFS) = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = **640 tracks**

4.2 SSTF (Shortest Seek Time First)

At each step, the head goes to the pending request **closest** to its current position. From 53, the closest is 65 (12 away), then 67 (2 away). From 67, the closest is 37 (30 away, closer than 98 at 31), then 14. Then it goes back up through 98, 122, 124 and 183: **53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183**

#	Move	Distance
1	53 → 65	|65 – 53| = 12
2	65 → 67	|67 – 65| = 2
3	67 → 37	|37 – 67| = 30
4	37 → 14	|14 – 37| = 23
5	14 → 98	|98 – 14| = 84
6	98 → 122	|122 – 98| = 24
7	122 → 124	|124 – 122| = 2
8	124 → 183	|183 – 124| = 59
Total head movement		**236 tracks**



Total head movement (SSTF) = 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = **236 tracks**

4.3 Conclusion

SSTF is much more efficient here: 236 tracks instead of 640, which is 404 tracks or about 63% less head movement. The charts show why. FCFS keeps jumping across the whole disk (98 → 183 → 37 → 122 → 14 → 124), while SSTF handles the nearby requests together and only makes one big trip. Since moving the head (seek time) is the slowest part of a disk access, less movement means faster average response and higher throughput.

SSTF has two weaknesses. It isn't always optimal (here it first goes down to 14 and then has to come all the way back up). More importantly, it can cause **starvation**: if new requests keep arriving near the head, a request far away, like 183, could wait for a very long time. FCFS is fair and never starves a request, but it's slow. In practice, systems often use SCAN or C-SCAN (the "elevator" algorithms), which keep most of SSTF's efficiency while guaranteeing every request is eventually served.